

DIGITAL ELECTRONICS

Unit-III: Map Simplification & Combinational Circuits

Comprehensive Study Notes

HPU BCA

Table of Contents

1. Minimization Techniques
2. Karnaugh Map (K-Map)
3. Sum of Products (SOP) Simplification
4. Product of Sums (POS) Simplification
5. Don't Care Conditions
6. Prime Implicants & Essential Prime Implicants
7. Quine-McCluskey Method
8. Combinational Circuits
9. Half Adder & Full Adder
10. Half Subtractor & Full Subtractor
11. Multiplexer (4:1 MUX) & Demultiplexer (1:4 DEMUX)
12. Encoder & Decoder
13. Questions

1. Minimization Techniques

1.1 Introduction to Minimization

Minimization is the process of simplifying complex Boolean expressions to their **most reduced form** using fewer logic gates, resulting in simpler, cheaper, and faster digital circuits.

Why Minimization is Important

Reason	Description
Cost Reduction	Fewer gates mean lower cost
Space Saving	Less circuit area required
Speed Improvement	Fewer gates mean faster operation
Power Consumption	Reduced power usage
Reliability	Fewer components mean less chance of failure

Techniques for Minimization

Technique	Description	Best For
Boolean Algebra	Using algebraic laws and theorems	Simple expressions
Karnaugh Map (K-Map)	Visual/graphical method	Up to 5-6 variables
Quine-McCluskey	Tabular method	More than 5 variables, automated simplification

2. Karnaugh Map (K-Map)

2.1 What is a Karnaugh Map?

A **Karnaugh Map (K-Map)** is a **pictorial method** of grouping together expressions with common factors and eliminating unwanted variables. It provides a systematic visual approach for simplifying Boolean expressions.

Key Characteristics

Feature	Description
Cell Arrangement	Uses Gray code so adjacent cells differ by only one variable
2 ⁿ Cells	For n variables, there are 2 ⁿ cells
Wrap-around	Top and bottom, left and right edges are adjacent
Visual Tool	Helps identify prime implicants and eliminate redundancies

Gray Code for K-Maps

```
2-Variable: 00 01 11 10
3-Variable: 000 001 011 010 110 111 101 100
4-Variable: 0000 0001 0011 0010 0110 0111 0101 0100
              1100 1101 1111 1110 1010 1011 1001 1000
```

Gray Code Rule: Only **one bit changes** between adjacent codes, ensuring logical adjacency in the map.

2.2 K-Map Structures

2-Variable K-Map (2² = 4 cells)

	B	
	0	1
A	0	1
	m0	m1
	m2	m3

Cells: $m0 = A'B'$, $m1 = A'B$, $m2 = AB'$, $m3 = AB$

3-Variable K-Map ($2^3 = 8$ cells)

	BC			
	00	01	11	10
A	0	1	1	0
	m0	m1	m3	m2
	m4	m5	m7	m6

Cells:

$m0 = A'B'C'$ $m1 = A'B'C$ $m3 = A'BC$ $m2 = A'BC'$
 $m4 = AB'C'$ $m5 = AB'C$ $m7 = ABC$ $m6 = ABC'$

4-Variable K-Map ($2^4 = 16$ cells)

		CD			
		00	01	11	10
AB	00	1	0	1	0
		m0	m1	m3	m2
	01	0	1	0	1
		m4	m5	m7	m6
	11	1	0	1	0
		m12	m13	m15	m14
	10	0	1	0	1
		m8	m9	m11	m10

Cells:

$m0=A'B'C'D'$ $m1=A'B'C'D$ $m3=A'B'CD$ $m2=A'B'CD'$
 $m4=A'BC'D'$ $m5=A'BC'D$ $m7=A'BCD$ $m6=A'BCD'$
 $m12=ABC'D'$ $m13=ABC'D$ $m15=ABCD$ $m14=ABCD'$
 $m8=AB'C'D'$ $m9=AB'C'D$ $m11=AB'CD$ $m10=AB'CD'$

Special Adjacencies: The four corners ($m0$, $m2$, $m8$, $m10$) are adjacent. Left-right and top-bottom edges wrap around.

2.3 Grouping Rules for K-Map

1. **Group sizes must be powers of 2** — 1, 2, 4, 8, 16 cells

2. **Groups should be as large as possible** — larger groups = fewer literals
3. **All 1s must be covered** — every 1 must be part of at least one group
4. **Groups can overlap** — overlapping groups may be necessary to cover all 1s
5. **Wrap-around is allowed** — groups can extend off the edges
6. **Avoid redundant groups** — don't include groups that only cover already-covered 1s

Grouping Hierarchy

Pair (2 cells) --> Eliminates 1 variable
 Quad (4 cells) --> Eliminates 2 variables
 Octet (8 cells) --> Eliminates 3 variables

2.4 Types of Groups

Pair (2 Adjacent Cells)

A **pair** is a group of **two adjacent 1s**. It eliminates **one variable**.

		BC			
		00	01	11	10
A	0	1	1		
	1				

Result: $F = A'C'$ (B is eliminated)

Quad (4 Adjacent Cells)

A **quad** is a group of **four adjacent 1s**. It eliminates **two variables**.

		BC			
		00	01	11	10
A	0	1	1	1	1
	1				

Result: $F = A'$ (B and C are eliminated)

Octet (8 Adjacent Cells)

An **octet** is a group of **eight adjacent 1s**. It eliminates **three variables**.

		CD			
		00	01	11	10
AB	00	1	1	1	1
	01				
	11				
	10				

Result: $F = B'C'$ (A and D are eliminated)

2.5 Don't Care Conditions

Don't care conditions are input combinations for which the output is not specified (can be either 0 or 1). They are marked as **X** in the K-Map and can be used to form larger groups for simpler expressions.

- Don't care conditions can be treated as **0 or 1** whichever helps in simplification
- They are not required to be covered
- They help create **larger groups** reducing the final expression

Example with Don't Care

$F(A,B,C,D) = \sum m(1,3,7,11,15)$ with don't care $d(0,2,5)$

		CD			
		00	01	11	10
AB	00	X	1	1	X
	01		X	1	
	11			1	
	10			1	

Using don't cares, we can form larger groups and get: $F = CD + A'D$

2.6 Prime Implicants & Essential Prime Implicants

Term	Definition
Prime Implicant	A group of adjacent 1s that cannot be expanded further (largest possible group)
Essential Prime Implicant	A prime implicant that covers at least one minterm not covered by any other prime implicant

Steps:

1. Identify all prime implicants (largest possible groups)
2. Identify essential prime implicants (must be included)
3. Select minimal set of prime implicants covering all 1s

3. Sum of Products (SOP) Simplification

3.1 What is Sum of Products?

The **Sum of Products (SOP)** form is a Boolean expression where **product terms** (ANDed literals) are ORed together.

General SOP Form

$$F = A'BC' + AB'C + ABC$$

Canonical SOP

Feature	Description
Minterms	Each product term must contain all variables in either complemented or uncomplemented form
Truth Table	Each 1 in truth table corresponds to a minterm
Sigma Notation	$F = \sum m(2, 5, 7)$ where numbers are minterm indices

Steps for SOP from K-Map

1. Identify all **1-cells** in the K-map
2. **Form groups** of adjacent 1s (powers of 2)
3. **Write product terms** for each group
4. **OR all product terms** together

3.2 Solved Examples: SOP Minimization

Example 1: $F(A, B, C) = \sum m(2, 3, 7)$

Step 1: Create K-Map

		BC				
		00	01	11	10	
A	0			1	1	(m2=010, m3=011)
	1			1		(m7=111)

Step 2: Form Groups

Group 1: m2, m3 --> A'B (C changes)

Group 2: m3, m7 --> BC (A changes)

Step 3: Write SOP Expression

$$F = A'B + BC$$

Example 2: $F(A, B, C) = \sum m(0, 1, 3, 2)$

		BC			
		00	01	11	10
A	0	1	1	1	1
	1				

All 4 cells in top row --> A' (eliminates B, C)

$$F = A'$$

Example 3: $F(A, B, C, D) = \sum m(0, 1, 3, 2, 4, 5, 7, 6)$

		CD			
		00	01	11	10
AB	00	1	1	1	1
	01	1	1	1	1
	11				
	10				

All 8 cells in top half --> A'

$$F = A'$$

Example 4: $F(A, B, C, D) = \sum m(0, 1, 3, 2, 8, 9, 11, 10)$

		CD			
		00	01	11	10
AB	00	1	1	1	1

01							
11							
10		1		1		1	

Group 1: Row 00 --> $A'B'$

Group 2: Row 10 --> AB'

$$F = A'B' + AB' = B'(A' + A) = B'$$

4. Product of Sums (POS) Simplification

4.1 What is Product of Sums?

The **Product of Sums (POS)** form is a Boolean expression where **sum terms** (ORed literals) are ANDed together.

General POS Form

$$F = (A + B' + C) * (A' + B + C)$$

Canonical POS

Feature	Description
Maxterms	Each sum term must contain all variables in either complemented or uncomplemented form
Truth Table	Each 0 in truth table corresponds to a maxterm
Pi Notation	$F = \prod M(1, 4, 6)$ where numbers are maxterm indices

Steps for POS from K-Map

1. Identify all **0-cells** in the K-map
2. **Form groups** of adjacent 0s (powers of 2)
3. Write sum terms for each group (variable = 0 → uncomplemented, variable = 1 → complemented)
4. **AND all sum terms** together

4.2 Solved Examples: POS Minimization

Example 1: $F(A, B, C) = \prod M(1, 4, 6)$

Step 1: Create K-Map with 0s for maxterms

		BC			
		00	01	11	10
A	0	1	0	1	1
	1	0	1	1	0

Group 1: m1 (001) alone --> $(A + B + C')$

Group 2: m4 (100) alone --> $(A' + B + C)$

Group 3: m6 (110) alone --> $(A' + B' + C)$

$$F = (A + B + C') * (A' + B + C) * (A' + B' + C)$$

Example 2: $F(A, B, C) = \prod M(0, 1, 3, 2)$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1				

All 4 cells in top row --> $F = A$

4.3 SOP vs POS Summary

Feature	SOP	POS
Form	$F = \sum(\text{minterms})$	$F = \prod(\text{maxterms})$
K-Map Focus	Group 1s	Group 0s
Implementation	AND-OR	OR-AND
Notation	$\sum m()$	$\prod M()$

5. Quine-McCluskey Method

The **Quine-McCluskey** method is a tabular technique for minimizing Boolean functions with **more than 5 variables** (where K-Maps become impractical). It is suitable for computer implementation.

Steps:

1. List all minterms with their binary representations
2. Group minterms by number of 1s
3. Combine adjacent groups (differ by one bit)
4. Repeat until no further combination is possible
5. Identify prime implicants
6. Select essential prime implicants using a prime implicant chart

6. Combinational Circuits

6.1 What is a Combinational Circuit?

A **combinational circuit** is a digital circuit where the **output depends only on the current input** values. There is no memory element, so past inputs do not affect the output.

Inputs ---> [LOGIC GATES] ---> Outputs
(A, B, C) (F)
Output at any time depends ONLY on current inputs

Characteristics

Characteristic	Description
No Memory	No storage elements
Instant Response	Output changes immediately when input changes
Time-independent	History of inputs does not matter
Function Representation	Can be expressed by a truth table
Implementation	Built using basic logic gates (AND, OR, NOT, NAND, NOR, XOR)
Examples	Adders, Multiplexers, Encoders, Decoders

6.2 Combinational vs Sequential Circuits

Feature	Combinational Circuit	Sequential Circuit
Memory	No memory	Has memory elements

Output	Depends only on current inputs	Depends on current inputs + previous state
Clock	Not required	Requires clock signal
Feedback	No feedback	Has feedback paths
Examples	Adder, Decoder, MUX	Flip-flops, Counters, Shift Registers

7. Half Adder

7.1 What is a Half Adder?

A **Half Adder** is a combinational circuit that **adds two single-bit binary numbers** (A and B) and produces a **Sum** and a **Carry** output.

```

A ---->|  HALF  |----> Sum (S)
B ---->|  ADDER  |----> Carry (C)
      _____
  
```

Truth Table

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions

```

Sum (S)  = A XOR B = A ⊕ B
Carry (C) = A AND B = A · B
  
```

Logic Circuit

```

A ---[XOR]--- Sum (S)
B ---[XOR]
      |
A ---[AND]--- Carry (C)
B ---[AND]
  
```

Limitations

- Cannot add carry from previous lower-bit addition
- Only adds two single-bit numbers
- Cannot be used directly for multi-bit addition

8. Full Adder

8.1 What is a Full Adder?

A **Full Adder** is a combinational circuit that **adds three single-bit binary numbers**: two input bits (A and B) and a **Carry-in** (Cin) from the previous lower bit addition.

```

A ---|  FULL  |
B ---|  ADDER  |----> Sum (S)
Cin -|-----|----> Carry (Cout)

```

Truth Table

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expressions

```

Sum (S) = A ⊕ B ⊕ Cin
Cout    = (A · B) + (Cin · (A ⊕ B))
        = (A · B) + (B · Cin) + (A · Cin)

```

Full Adder using Two Half Adders

```

A ---[HALF ADDER]--- Sum1 ---[HALF ADDER]--- Sum
B ---[HALF ADDER]      |
                        |--- Carry1 ----[HALF ADDER]
Cin -----[HALF ADDER]
                        |--- Cout
                        Carry2 ---|
                        Carry1 ---[OR]--- Cout
                        Carry2 ---[OR]

```

9. Half Subtractor

9.1 What is a Half Subtractor?

A **Half Subtractor** is a combinational circuit that **subtracts two single-bit binary numbers** (A - B) and produces a **Difference** and a **Borrow** output.

Truth Table

A	B	Difference (D)	Borrow (Bout)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Expressions

Difference (D) = $A \oplus B$
Borrow (Bout) = $A' \cdot B$

10. Full Subtractor

10.1 What is a Full Subtractor?

A **Full Subtractor** is a combinational circuit that **subtracts three single-bit binary numbers**: minuend (A), subtrahend (B), and **Borrow-in** (Bin) from the previous lower bit subtraction.

```

A ---|  FULL      |
B ---|  SUBTRACTOR |----> Difference (D)
Bin -|_____ |----> Borrow (Bout)
  
```

Truth Table

A	B	Bin	Difference (D)	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Boolean Expressions

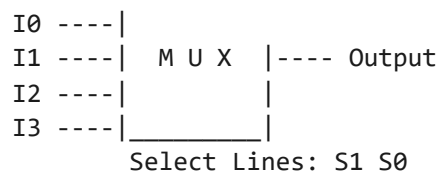
```

Difference (D) = A ⊕ B ⊕ Bin
Borrow (Bout) = (A' · B) + (Bin · (A ⊕ B))
  
```

11. Multiplexer (MUX)

11.1 What is a Multiplexer?

A **Multiplexer (MUX)** is a combinational circuit that **selects one input** from multiple inputs and forwards it to a single output based on select lines.



4:1 Multiplexer

A 4:1 MUX has 4 inputs, 2 select lines (S1, S0), and 1 output.

Truth Table

S1	S0	Output
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Boolean Expression

$$\text{Output} = S1' S0' I0 + S1' S0 I1 + S1 S0' I2 + S1 S0 I3$$

11.2 Demultiplexer (DEMUX)

A **Demultiplexer (DEMUX)** performs the reverse operation of a MUX — it takes a single input and routes it to one of several outputs based on select lines.

1:4 Demultiplexer

S1	S0	Y0	Y1	Y2	Y3
0	0	Input	0	0	0
0	1	0	Input	0	0
1	0	0	0	Input	0
1	1	0	0	0	Input

12. Encoder

12.1 What is an Encoder?

An **Encoder** is a combinational circuit that **converts multiple inputs** into a coded binary output. It performs the inverse operation of a decoder.

4:2 Encoder

I3	I2	I1	I0	O1	O0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Boolean Expressions

$$O0 = I1 + I3$$

$$O1 = I2 + I3$$

13. Decoder

13.1 What is a Decoder?

A **Decoder** is a combinational circuit that **converts binary input** into a unique output line. For n inputs, there are 2^n outputs.

2:4 Decoder

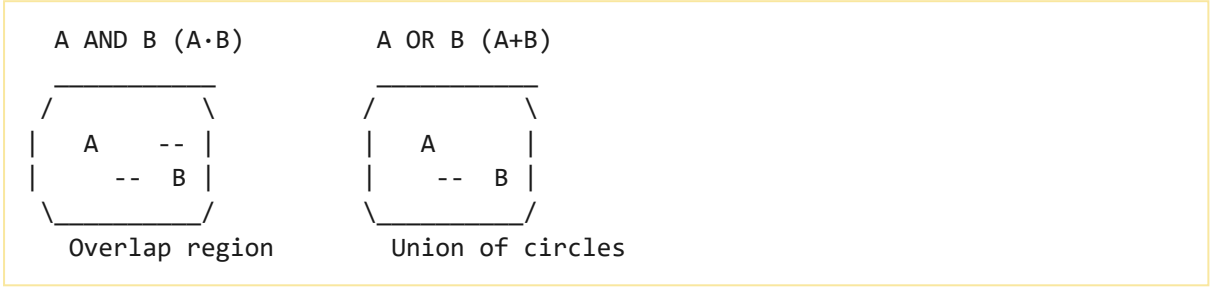
I1	I0	O3	O2	O1	O0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

3:8 Decoder

I2	I1	I0	O7	O6	O5	O4	O3	O2	O1	O0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

14. Venn Diagrams for Boolean Operations

Venn diagrams provide a visual representation of Boolean operations using overlapping circles.



Boolean Operation	Venn Diagram Representation
$A \cdot B$ (AND)	Overlap region between circles A and B
$A + B$ (OR)	Union of circles A and B
A' (NOT)	Outside circle A
$A \oplus B$ (XOR)	Union minus overlap

15. Quick Revision

K-Map Grouping Summary

Group	Cells	Eliminated Variables	Example Result
Pair	2	1 variable	$A'B + A'B' = A'$
Quad	4	2 variables	$A'B'C + A'BC = A'C$
Octet	8	3 variables	Eliminates 3 variables

Key Formulas

Formula	Description
$F = \sum m()$	Sum of Products notation
$F = \prod M()$	Product of Sums notation
2^n	Number of K-map cells for n variables
$Sum = A \oplus B$	Half Adder Sum
$Carry = A \cdot B$	Half Adder Carry
$Sum = A \oplus B \oplus C_{in}$	Full Adder Sum
4:1 MUX Output	$S1'S0'I0 + S1'S0'I1 + S1S0'I2 + S1S0I3$

16. HPU Important Questions

Short Answer Questions (2-5 Marks)

1. What is minimization of Boolean functions?
2. What is a Karnaugh map (K-map)?
3. What is the advantage of using K-map over Boolean algebra?
4. What is Sum of Products (SOP) form?
5. What is Product of Sums (POS) form?
6. What are minterms and maxterms?
7. What is the difference between a pair, quad, and octet?
8. What are prime implicants and essential prime implicants?
9. What is a combinational circuit? Give an example.
10. What is the difference between combinational and sequential circuits?
11. What is a multiplexer? Explain 4:1 MUX.
12. What is the difference between an encoder and a decoder?

Long Answer Questions (10 Marks)

1. Explain the K-map minimization technique for Boolean functions up to 4 variables with examples.
2. Explain SOP and POS forms with examples and their K-map implementation.
3. Explain Venn diagrams for Boolean operations.
4. What are combinational circuits? Explain Half Adder and Full Adder with truth tables and logic diagrams.
5. Explain Full Subtractor with truth table and logic diagram.
6. Explain 4:1 Multiplexer and 1:4 Demultiplexer with truth tables and expressions.
7. Explain Encoder (4:2) and Decoder (2:4) with truth tables.
8. Differentiate between combinational and sequential circuits.

Objective Questions (1 Mark)

1. A Karnaugh map for 3 variables has: (a) 4 cells (b) 8 cells (c) 16 cells (d) 32 cells

Answer: (b)

2. A 4-variable K-map has how many cells? (a) 4 (b) 8 (c) 16 (d) 32

Answer: (c)

3. A quad in K-map eliminates how many variables? (a) 0 (b) 1 (c) 2 (d) 3

Answer: (c)

4. An octet in K-map eliminates how many variables? (a) 0 (b) 1 (c) 2 (d) 3

Answer: (d)

5. The output of a combinational circuit depends on: (a) Current inputs only (b) Past inputs only (c) Both (d) None

Answer: (a)

6. Which circuit has memory? (a) Combinational (b) Sequential (c) Both (d) None

Answer: (b)

7. How many select lines does a 4:1 MUX have? (a) 1 (b) 2 (c) 3 (d) 4

Answer: (b)

8. A Half Adder produces: (a) Sum only (b) Carry only (c) Sum and Carry (d) Difference

Answer: (c)

DIGITAL ELECTRONICS — Unit III: Map Simplification & Combinational Circuits

HPU BCA Study Notes